CSC 548 HW4: Distributed Spark SpMV Architecture Report

Student Unity ID: fclin

Course: CSC 548 Parallel Systems (Fall 2026)

Execution Environment: NC State ARC Cluster (Spark 3.5.0, Python 3.11, OpenJDK 8)

Allocation: 16 Cores, 16 GB Driver Heap

1. Executive Summary & Sparse Matrix Structural Profiles

Sparse Matrix-Vector Multiplication (SpMV) computes $y = A \cdot x$. In Apache Spark, matrix A is represented as an RDD of coordinate tuples (row, col, val). Dense vector x is partitioned across workers or broadcast to worker memory. We evaluate six benchmark matrices from the SuiteSparse collection. Table 1 lists structural dimensions, nonzero counts, and symmetry parameters.

Matrix	Rows (m)	Cols (n)	Base NNZ	Expanded NNZ	Size	Symmetry	Structural Profile
D6-6	120,576	23,740	146,880	146,880	2.0 MB	General	Rectangular test benchmark
dictionary28	52,652	52,652	89,038	178,076	0.9 MB	Symmetric	High-sparsity word graph
bfly	49,152	49,152	98,304	196,608	1.1 MB	Symmetric	Regular butterfly topology
roadNet-CA	1,971,281	1,971,281	2,766,607	5,533,214	39.2 MB	Symmetric	Planar road network, large m
Ga3As3H12	61,349	61,349	3,016,148	5,970,947	90.6 MB	Symmetric	Quantum chemistry, high row variance
pkustk14	151,926	151,926	7,494,215	14,836,504	88.0 MB	Symmetric	Stiffness matrix, high density

Table 1: Structural dimensions, nonzero counts, and storage metrics of the benchmark suite.

2. Implementation Design & DAG Lineage Analysis

We implemented two distributed SpMV execution strategies in PySpark: (1) Column Shuffle Join (spmv_join) and (2) Broadcast Lookup (spmv_broadcast).

Scheme 1: Column Shuffle Join (Two-Shuffle DAG). The input matrix RDD is re-keyed by column: (col, (row, val)). Vector x is parallelized into an RDD of (col, x[col]). The two RDDs undergo a key-based join across the network. A subsequent map calculates partial scalar products (row, val * x[col]). Finally, a reduceByKey sums partial products into the output vector y. This pipeline executes two wide-dependency shuffle boundaries (one for column joining and one for row aggregation).

Scheme 2: Broadcast Lookup (Single-Shuffle DAG). Dense vector x fits into worker memory (15.8 MB for the largest matrix, roadNet-CA). The driver broadcasts x via sc.broadcast(x). Workers access x_broadcast.value locally inside a single map transformation. This computes (row, val * x[col]) without inter-worker data exchange. Only the terminal reduceByKey triggers a network shuffle. Broadcast eliminates Stage 1 network shuffle traffic entirely.

3. Partitioning & Network Shuffle Study

Table 2 reports the measured execution times, stage counts, and shuffle bytes from the ARC cluster parameter sweep across partition counts $p \in \{1, 2, 4, 8, 16, 32\}$.

Shuffle Stage Origin Analysis: In the join scheme, the column join incurs wide dependencies because matrix nonzeros with identical column indices reside on different worker partitions. The join shuffles all NNZ entries across the network. Conversely, the broadcast scheme delivers vector x to worker heaps once via BitTorrent-based broadcast. Workers perform map-side multiplication locally. The only remaining shuffle is the terminal reduceByKey, where partial products sharing the same row index are merged. Consequently, broadcast achieves an immediate $2.5\times\text{--}4.2\times$ speedup over the join baseline.

4. Performance & Cross-Model Paradigm Analysis (HW1–HW4)

Figure 4 and Table 3 compare optimal SpMV runtimes on identical SuiteSparse matrices across all four course programming paradigms: MPI (HW1), OpenMP (HW2), CUDA (HW3), and Apache Spark (HW4).

Why Spark SpMV is 10x–1000x Slower than Native MPI/CUDA:

1. **DAG Scheduling & Task Dispatch Latency:** Native C/MPI and CUDA kernels launch with microseconds of overhead ($<5\ \mu\text{s}$). Spark must construct DAG execution graphs, serialize Task objects, submit jobs to executor thread pools, and manage task lifecycle events. This scheduling floor introduces $10\text{--}30\text{ ms}$ of fixed latency per stage regardless of matrix size.

2. **Py4J IPC & Python Boxing Overhead:** In PySpark, worker processes communicate with the JVM via local TCP sockets. Every coordinate tuple (row, col, val) undergoes Python object allocation, pickle serialization, socket transmission, JVM byte decoding, and reverse unboxing. In contrast, OpenMP and CUDA process raw contiguous FP32/FP64 memory arrays directly in hardware registers.

3. **Memory Bandwidth & Cache Hierarchy:** Native CUDA kernels achieve up to 605 GB/s of effective memory bandwidth on ARC's Tesla V100 GPU by leveraging 128-byte coalesced global memory loads and hardware shared memory. Spark tuples are non-contiguous heap objects scattered in JVM memory, destroying CPU L1/L2 data cache locality.

4. **Batch ETL Design vs. Fine-Grained Linear Algebra:** Apache Spark is engineered for high-throughput batch ETL, fault tolerance, and disk spillover over commodity clusters. Fine-grained numeric kernels like SpMV are dominated by memory access latency and tight arithmetic loops, making Spark's fault-tolerant abstraction heavily mismatched for high-performance linear algebra.